

SWAP

Proyecto final:

Servicios de almacenamiento distribuido de bajo coste

Número de trabajo: 32

Aproximadamente 11 horas de trabajo

Fernando Castilla Roperro

Álvaro García Zafra



Índice:

1. Introducción
2. Antecedentes o preliminares
3. Objetivos
4. Desarrollo y/o análisis
5. Conclusiones
6. Bibliografía

1. Introducción

Hoy en día, todo el mundo genera y comparte archivos constantemente: fotos, documentos, vídeos, copias de seguridad... Y con tanto contenido digital dando vueltas, es fácil imaginar que necesitamos formas inteligentes de guardarlo todo sin depender de un solo ordenador o servidor que pueda fallar. ¿Qué pasaría si ese servidor se rompe? ¿Y si se queda corto de espacio?

Para dar respuesta a este tipo de situaciones, se han desarrollado sistemas de almacenamiento distribuido, pero...¿Qué es el almacenamiento distribuido? Bien, citando una descripción literal de nutanix.com “El almacenamiento distribuido es un sistema de almacenamiento definido por software que permite el acceso a los datos, en cualquier momento, desde cualquier lugar y solo a aquellas personas que queramos que accedan”, explicado por nosotros; no son más que formas de guardar archivos repartidos entre varios equipos o servicios, de forma que si uno falla, los datos siguen estando a salvo en otros. Y lo mejor: no hace falta gastar una fortuna para tener algo así.

Este trabajo nace con esa idea: crear un ejemplo de solución simple y económica para almacenar archivos en distintos servidores o contenedores, aprovechando tecnologías libres como Docker, Nginx y PHP, hemos diseñado una pequeña aplicación que permite subir archivos desde un navegador web.



2. Antecedentes o preliminares

Para poder construir un sistema de almacenamiento distribuido sencillo y eficaz, es necesario apoyarse en tecnologías que simplifiquen la gestión de servidores y que permitan distribuir la carga entre ellos. En este trabajo, se han utilizado dos herramientas en las que nos hemos ido formando durante el curso: Docker y Nginx.

Sobre Docker, en el sitio web de amazon nos cuentan que Docker es una plataforma de software que le permite crear, probar e implementar aplicaciones rápidamente. Docker empaqueta software en unidades estandarizadas llamadas contenedores que incluyen todo lo necesario para que el software se ejecute, incluidas bibliotecas, herramientas de sistema, código y versión ejecutable. Nosotros hemos usado estos contenedores como sitios web en donde almacenar los archivos.

Sobre Nginx, en hostinger.com nos lo introducen muy bien como un famoso software de servidor web de código abierto. En su versión inicial, funcionaba en servidores web HTTP. Sin embargo, hoy en día también sirve como proxy inverso, balanceador de carga HTTP y proxy de correo electrónico para IMAP, POP3 y SMTP. En nuestro caso, lo hemos usado como balanceador de carga entre los contenedores.

Por último, vamos a hablar un poco sobre MySQL, según la página oficial de Oracle; MySQL es un sistema de gestión de bases de datos relacionales (RDBMS) de código abierto que se utiliza para almacenar y gestionar datos. Nosotros lo hemos usado como base de datos para gestionar el almacenamiento de los archivos en los contenedores.



3. Objetivos

Se pretende desarrollar un sistema de almacenamiento distribuido que permita ofrecer servicios de almacenamiento. Para ello, hemos subdividido el trabajo en las siguientes subtarear:

1. Aprovechar la arquitectura de contenedores basada en Docker desarrollada durante el curso para alojar múltiples servidores de almacenamiento independientes.
 2. Aprovechar, a su vez; el sistema de balanceo de carga con Nginx para que distribuya las peticiones de subida de archivos entre los distintos servidores disponibles.
 3. Desarrollar un script en PHP que nos permita subir archivos a los servidores y registrar la información correspondiente en una base de datos.
 4. Crear una base de datos relacional para almacenar información sobre los archivos subidos, como el nombre, ruta, fecha y el contenedor donde se guardaron.
 5. Facilitar el acceso a la lista de archivos subidos mediante una interfaz sencilla en PHP.
 6. Asegurar que el sistema sea fácilmente replicable, escalable y comprensible para futuras ampliaciones o adaptaciones.
 7. Documentar el proceso de desarrollo, así como las decisiones técnicas tomadas, para dejar constancia del funcionamiento del sistema.
- 

4. Desarrollo y/o análisis

Vamos a abordar los objetivos de desarrollo de la aplicación, es decir; los objetivos desde el 1. al 5. Como ya hemos mencionado, para los objetivos 1. y 2. hemos reutilizado el trabajo que ya hemos realizado durante el curso, por lo que la estructura de los servidores web y del balanceador Nginx permanece siendo la misma.

```
web8:
  build:
    context: ./AlmDistAF-apache
    dockerfile: DockerFileApache
  image: almdistaf-apache-image:pf
  container_name: web8
  volumes:
    - ./AlmDistAF-apache/web_AlmDistAF:/var/www/html
    - ./AlmDistAF-certificados:/etc/apache2/ssl
    - web8_archivos:/var/www/html/archivos
  networks:
    red_web:
      ipv4_address: 192.168.10.18
  cap_add:
    - NET_ADMIN

Run Service
balanceador-nginx:
  build:
    context: ./AlmDistAF-nginx
    dockerfile: DockerFileNginx
  image: almdistaf-nginx-image:pf
  container_name: balanceador-nginx
  ports:
    - "8443:443"
  volumes:
    - ./AlmDistAF-nginx/AlmDistAF-nginx-ssl.conf:/etc/nginx/nginx.conf
    - ./AlmDistAF-certificados:/etc/nginx/ssl
  networks:
    red_web:
      ipv4_address: 192.168.10.50
  depends_on:
    - web1
    - web2
    - web3
    - web4
    - web5
    - web6
    - web7
    - web8
```

(3.)Dentro de la carpeta web de los servidores, desde el index.php vamos a permitir que cualquier usuario pueda interactuar con el sistema sin conocimientos técnicos. En solo unos clics, se puede subir un archivo y comprobar que ha quedado registrado correctamente. Para ello, vamos a hacer una interfaz sencilla con un botón para seleccionar archivo, otro botón para subirlo y un enlace hacia la base de datos con los archivos subidos.

```
<body>
  <h2>Subir un archivo</h2>
  <form action="subir.php" method="post" enctype="multipart/form-data">
    <input type="file" name="archivo">
    <input type="submit" value="Subir">
  </form>

  <p><a href="mostrar.php">Ver archivos subidos</a></p>
</body>
```

El elemento central del archivo es un formulario (<form>) que permite al usuario seleccionar un archivo desde su dispositivo y enviarlo al servidor. action="subir.php" indica que, al enviar el formulario; los datos se procesarán en el script subir.php. Los datos se enviarán por POST, ideal para transferencias de archivos.

enctype="multipart/form-data" es obligatorio cuando se envían archivos binarios. Dentro del formulario:

Vamos a desglosar lo que hacemos en el subir.php, paso por paso:

1. Utilizamos la función gethostname() para identificar el nombre del servidor (contenedor Docker) que ha recibido el archivo. Esto es útil en un entorno distribuido, ya que nos permite saber en qué contenedor se encuentra cada archivo.

```
$contenedor = gethostname();
```

2. Definimos una carpeta local llamada archivos/ donde se guardarán todos los archivos subidos. Si la carpeta no existe, se crea automáticamente con permisos adecuados.

```
if (!is_dir($dir_subida)) {

    mkdir($dir_subida, 0777, true);
```

3. Se comprueba si hubo algún error durante la subida del archivo. Si ocurre un error, se muestra un mensaje y se detiene la ejecución del script.

```
if ($_FILES['archivo']['error'] !== UPLOAD_ERR_OK) {  
  
    echo "Error en la subida: Código " . $_FILES['archivo']['error'] .  
    "<br>";  
  
    exit;  
  
}
```

4. Se toma el archivo temporal recibido y se mueve a la carpeta archivos/, manteniendo su nombre original. Si se mueve correctamente, se muestra un mensaje de éxito.

```
move_uploaded_file($_FILES['archivo']['tmp_name'], $fichero_subido)
```

5. Se establece una conexión con el servidor de base de datos MySQL. Los datos de conexión (host, usuario, contraseña y base de datos) están definidos explícitamente.

```
$conn = new mysqli($servername, $username, $password, $dbname);
```

6. Se registra en la base de datos el nombre del archivo, su ruta y el nombre del contenedor en el que ha sido almacenado. Esto permite consultar después desde qué servidor se sirvió cada archivo.

```
$sql = "INSERT INTO archivos (nombre, ruta, contenedor) VALUES  
('$nombreArchivo', '$rutaArchivo', '$nombreContenedor')";
```

7. Tras subir y registrar el archivo, se ofrece al usuario un mensaje de éxito y un enlace para volver al formulario inicial (index.php), facilitando nuevas subidas.

```
echo '<p><a href="index.php">Volver al formulario</a></p>';
```



A continuación, presentamos el archivo completo:

```
$dir_subida = 'archivos/';
$contenedor = gethostname(); // El nombre del contenedor actual

echo "Servidor actual: " . $contenedor . "<br>";

if (!is_dir(filename: $dir_subida)) {
    mkdir(directory: $dir_subida, permissions: 0777, recursive: true);
}

if ($FILES['archivo']['error'] !== UPLOAD_ERR_OK) {
    echo "Error en la subida: Código " . $_FILES['archivo']['error'] . "<br>";
    exit;
}

$fichero_subido = $dir_subida . basename(path: $_FILES['archivo']['name']);

if (move_uploaded_file(from: $_FILES['archivo']['tmp_name'], to: $fichero_subido)) {
    echo "Archivo subido correctamente a " . $fichero_subido . "<br>";

    // Conectar a la base de datos
    $servername = "mysql";
    $username = "archivos_user";
    $password = "archivos_pass";
    $dbname = "archivos_db";

    $conn = new mysqli(hostname: $servername, username: $username, password: $password, database: $dbname);

    if ($conn->connect_error) {
        die("Conexión fallida: " . $conn->connect_error . "<br>");
    }

    $nombreArchivo = $conn->real_escape_string(string: basename(path: $_FILES['archivo']['name']));
    $rutaArchivo = $conn->real_escape_string(string: $fichero_subido);
    $nombreContenedor = $conn->real_escape_string(string: $contenedor);

    $sql = "INSERT INTO archivos (nombre, ruta, contenedor) VALUES ('$nombreArchivo', '$rutaArchivo', '$nombreContenedor')";

    if ($conn->query(query: $sql) === TRUE) {
        echo "Registro guardado en base de datos.<br>";
    } else {
        echo "Error al guardar registro: " . $conn->error . "<br>";
    }

    $conn->close();
} else {
    echo "Error al mover el archivo.<br>";
}
```

(4.) Para la base de datos, como hemos mencionado previamente; hemos usado MySQL y la hemos implementado como un contenedor más en Docker, dentro del docker compose:

```
mysql:
  image: mysql:5.7
  container_name: mysql
  restart: always
  environment:
    MYSQL_ROOT_PASSWORD: rootpass
    MYSQL_DATABASE: archivos_db
    MYSQL_USER: archivos_user
    MYSQL_PASSWORD: archivos_pass
  volumes:
    - mysql_data:/var/lib/mysql
    - ./mysql-init:/docker-entrypoint-initdb.d
  networks:
    - red_servicios
  ports:
    - "3306:3306"
```

Respecto a la tabla, la hemos declarado con una clave primaria ID, y atributos básicos para lo que queremos almacenar como el nombre del archivo, la ruta, el contenedor en el que se ha almacenado y la fecha de subida.

```
1  CREATE DATABASE IF NOT EXISTS archivos_db;
2  USE archivos_db;
3
4  CREATE TABLE IF NOT EXISTS archivos (
5    id INT AUTO_INCREMENT PRIMARY KEY,
6    nombre VARCHAR(255) NOT NULL,
7    ruta VARCHAR(255) NOT NULL,
8    contenedor VARCHAR(100) NOT NULL,
9    fecha_subida TIMESTAMP DEFAULT CURRENT_TIMESTAMP
10 );
```

(5.) Por último, para facilitar el acceso a los archivos subidos hemos implementado un archivo mostrar.php que accede a la base de datos e imprime una tabla con los datos registrados en la misma. El archivo en concreto es:

```
<?php
$servername = "mysql";
$username = "archivos_user";
$password = "archivos_pass";
$dbname = "archivos_db";

// Crear conexión
$conn = new mysqli(hostname: $servername, username: $username, password: $password, database: $dbname);

// Comprobar conexión
if ($conn->connect_error) {
    die("Conexión fallida: " . $conn->connect_error);
}

$sql = "SELECT nombre, ruta, contenedor, fecha_subida FROM archivos ORDER BY fecha_subida DESC";
$result = $conn->query(query: $sql);

echo "<h1>Archivos Subidos</h1>";
echo "<table border='1' cellpadding='5'><tr><th>Nombre</th><th>Ruta</th><th>Contenedor</th><th>Fecha de subida</th></tr>";

if ($result->num_rows > 0) {
    while($row = $result->fetch_assoc()) {
        echo "<tr>
            <td>" . htmlspecialchars(string: $row["nombre"]) . "</td>
            <td>" . htmlspecialchars(string: $row["ruta"]) . "</td>
            <td>" . htmlspecialchars(string: $row["contenedor"]) . "</td>
            <td>" . $row["fecha_subida"] . "</td>
        </tr>";
    }
} else {
    echo "<tr><td colspan='4'>No hay archivos subidos aún.</td></tr>";
}

echo "</table>";

$conn->close();

echo '<p><a href="index.php">Volver</a></p>';
?>
.....
```

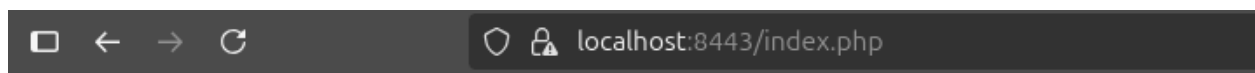
5. Conclusiones

Finalmente, vamos a ver los resultados de la aplicación y cómo ejecutarla.

Primero, debemos crear la red y lanzar los contenedores; para ello daremos permisos de ejecución a los scripts crear_red_web.sh y launch.sh y los lanzamos en ese orden:

```
ferrjr@ferrjrubu:~/Escritorio/ProSWAP$ ./crear_red_web.sh
✓ La red 'red_web' ya existe. No se hace nada.
ferrjr@ferrjrubu:~/Escritorio/ProSWAP$ ./launch.sh
Iniciando despliegue de contenedores...
```

Para acceder al sistema lo haremos desde <https://localhost:8443/> ahí veremos la siguiente interfaz:



Subir un archivo

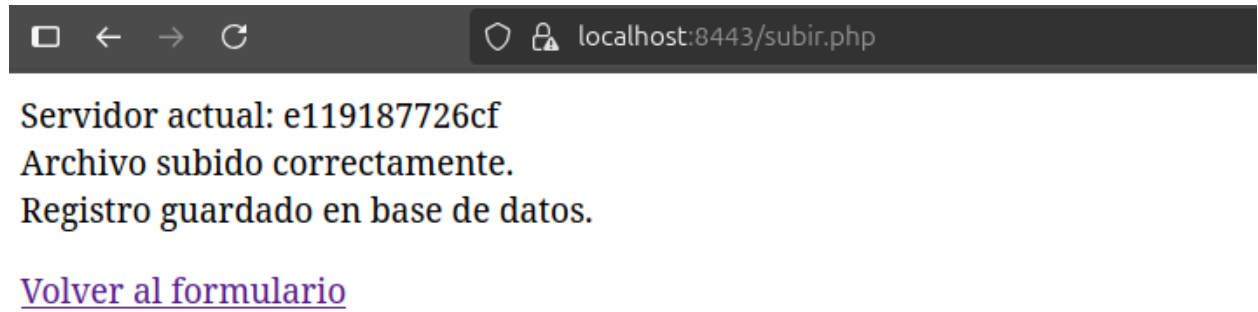
Examinar... audio.wav

Subir

[Ver archivos subidos](#)

Como se puede observar, el funcionamiento es interactivo y fácil de entender, veamos que pasa al subir un archivo:





Y, si accedemos a la lista de archivos subidos:

A screenshot of a web browser window. The address bar shows 'localhost:8443/mostrar.php'. The page has a heading 'Archivos Subidos' followed by a table with four columns: 'Nombre', 'Ruta', 'Contenedor', and 'Fecha de subida'. The table contains three rows of data.

Nombre	Ruta	Contenedor	Fecha de subida
audio.wav	/archivos/audio.wav	e119187726cf	2025-06-07 16:43:42
Sin título.png	/archivos/Sin título.png	ab61078cefaa	2025-06-07 16:43:18
Rosa1.jpg	/archivos/Rosa1.jpg	1b34ff76ae84	2025-06-07 16:43:12

En conclusión, podemos resumir nuestro trabajo como una arquitectura web distribuida basada en contenedores Docker, compuesta por múltiples servidores Apache balanceados mediante Nginx, con comunicación segura mediante certificados SSL y un sistema de almacenamiento para el intercambio de archivos.

La solución implementada ha demostrado ser funcional, permitiendo que cualquier cliente pueda interactuar con los servidores de forma transparente y segura.

Como posibles mejoras para trabajos futuros, podría integrarse un sistema de autenticación centralizada, la gestión dinámica de certificados y la persistencia de sesiones entre nodos mediante un backend compartido. Todo ello contribuiría a una mayor robustez, seguridad y eficiencia en entornos de producción reales.

6. Bibliografía

<https://www.nutanix.com/es/info/distributed-storage>

<https://www.hostinger.com/es/tutoriales/que-es-nginx>

<https://aws.amazon.com/es/docker/>

<https://www.oracle.com/es/mysql/what-is-mysql/>